Agent Traces with MLFLow and Red Hat OpenShift AI

Version 1, 2026-07-02

Home

Goal

This course teaches how to deploy MLFlow, as included with Red Hat OpenShift AI 3.4; How to collect traces of an agentic application, using the MLFlow SDK with Python; and how to visualize those traces using the Red Hat OpenShift AI web interface.

This course is intended to require learners between 4 hours to one working day to complete.

Work In Progress

Objectives

On completing this course, you should be able to:

- Understand how MLFlow relates to agentic applications and to Red Hat OpenShift AI.
- Deploy a development environment with MLFlow
- Collect traces from an agentic application using MLFlow

Audience

- Software developers working on agentic applications and agentic harnesses.
- Red Hat and Partner roles that perform demonstrations, PoC and PoV of MLFlow with Red Hat OpenShift AI, such as Solutions Architects.
- Red Hat and Partner roles that implement and support MLFlow with Red Hat OpenShift AI, such as Services Consultants, Technical Account Managers, and Customer Support Engineers.

Prerequisites

This course assumes that you have the following experience:

- Familiarity with generative AI and agentic application concepts.
- Familiarity with the basic operation of Red Hat OpenShift AI.
- Python programming skills are recommended but not required, because this training provides sample code which is ready to run.

Classroom environment

The demo catalog entry Red Hat OpenShift AI 3.4 provides an SNO OpenShift cluster, backed by an AWS instance with a single GPU, with a predeployed llama-32-3b-instruct model which is capable of making tool calls.

This course uses that demo entry as-is, and any additional components which are required by either MLFlow itself or any of the course activities will be configured by learners, as part of course activities, by using sample Python code and Kubernetes manifests provided in a public Git repository.

Other sources of information

For documentation about MLFlow with Red Hat OpenShift AI, see the Red Hat OpenShift AI product documentation.

For upstream MLFlow documentation, see the MLFlow project documentation.

Author

Fernando Lozano
Training Content Architect
Red Hat - Product Portfolio Marketing & Learning

Lab Environment

Instructions to Launch Your Lab on the Red Hat Demo Platform (RHDP)

1. Log in to the RHDP portal
2. Search for the catalog entry: `Red Hat Openshift AI 3`
3. Click the catalog entry in the search results.
4. On the catalog page, click **Order**.
5. Fill out the required details in the order form and accept the defaults for other fields. You can use the smaller AWS instance type.
6. Review the warning at the bottom of the form and check:
I confirm that I understand the above warnings.
7. Await the email notification stating that your demo environment is ready and providing its access credentails and URLs.

Important Notes:

- This lab may take approximately **1:30** hours to become ready.
- You can also retrieve access information directly from the RHDP portal.
- This lab uses GPU-enabled instances on AWS, which are in short supply. It require multiple attempts to provision or restart.

> Do **not** follow the instructions/showroom from the demo catalog entry. This course reuses that demo but does not require that you perform any of its activities.
>
> You will use the preprovisioned instance of Red Hat OpenShift AI as-is, and deploy MLFlow and sample agentic applications as part of this course activities.

How to Access Your Lab via the RHDP Portal:

1. On the RHDP portal, click **Services** option in the left-hand menu.
2. Select your lab from the services listings on the right-hand side of the page to view access details.

RHDP Portal Links

- RedHat associates: https://demo.redhat.com/
- RedHat partners: https://partner.demo.redhat.com/

Understanding MLFlow and Red Hat OpenShift AI

Goal

Understand how MLFlow relates to agentic applications and to Red Hat OpenShift AI.

This chapter introduces MLFlow in the context of Red Hat OpenShift AI, focusing on features relevant to generative AI and agentic applications. You'll learn about MLFlow's architecture, supported features, and how it integrates with RHOAI components.

Introduction to MLFlow in Red Hat OpenShift AI

*Estimated reading time: **5 minutes**.*

Objective

Understand MLFlow's features for agentic applications and how it integrates with Red Hat OpenShift AI.

Not started yet.

MLFlow Features in RHOAI

Lorem ipsum

Focus on Generative AI Applications

Lorem ipsum

MLFlow Experiments

Lorem ipsum

What's Next

Now that you understand MLFlow's role in RHOAI and agentic applications, the next section provides a quiz to verify your understanding of key MLFlow concepts.

Quiz: MLFlow Concepts Review

*Estimated reading time: **5 minutes**.*

Objective

Verify your understanding of MLFlow concepts and terminology.

Interactive Assessment

What's next

Now that you've verified your understanding of MLFlow concepts, the next chapter explores how to deploy MLFlow in your RHOAI environment for development purposes.

Lab: Explore the RHOAI Instance

Objective

Explore the predeployed Red Hat OpenShift AI instance to familiarize yourself with its configuration and with the new Gen AI Studio feature from 3.x releases.

> Pending review

Before you begin

You need access to the predeployed Red Hat OpenShift AI (RHOAI) instance provided in your lab environment. Ensure you have:

- Access credentials for the RHOAI dashboard
- A web browser to access the RHOAI dashboard

Instructions

In this lab, you will log in to the RHOAI dashboard and explore its main features and components to understand the platform's capabilities.

1. Access the RHOAI Dashboard.
   a. Navigate to the RHOAI dashboard URL of your demo environment.
   b. Log in using the `admin` user, from the credentials provided for your demo environment.
2. Explore the precreated project.

   Your RHOAI instance contains a model which is already deployed in a regular project named `my-first-model` — which is **not** an AI project.

   That model takes up all GPU capacity available to your RHOAI instance, so any change to the model deployment requires that you stop it and restart after making changes.

   a. In the left pane, select **AI hub › Models** and then select the **Deployments** tab.
   b. In the **Project** field, type `my` and select `my-first-model`.

   > Depending on the current page of the RHOAI dashboard, it may be necessary to first select `All projects` before you can search and select the `my-first-model` project.

c. Verify that there is a model deployment named `llama-32-3b-instruct` and its status is `Ready`.

d. Click **View**, under the **Endpoints** column for the model row, and verify that it provides only an internal endpoint. This means the model cannot be accessed from outside of its OpenShift cluster, unless you change its model deployment.

3. Try the model with a Gen AI Studio Playground.

a. In the left pane, select **Gen AI studio › Playground**.

b. In the **Project** field, type `my` and select `my-first-model`.

> The project previously selected in one page of the RHOAI dashboard may not be the current selection in another page of the RHOAI dashboard.

c. If you do not see a **Configure** pane, click **Settings**. On the **Configure** pane, select the **Model** tab and verify that the selected model is named `llama-32-3b-instruct`. Ignore the prefix on the model name.

d. In the right pane, click the `Send a message…` text and type a prompt, for example:

```
What is RHEL?
```

e. Type `Enter` to submit the prompt and see the response from the model. You may get different answers, if you attempt the same prompt again, but they all would, hopefully, be correct and reasonable.

4. Verify that there are no RHOAI dasbboard pages related to MLFlow.

a. In the left pane, expand the **Gen AI studio** category, which contains entries named **Playground** and **AI asset endpoints**.

After you enable MLFlow, there will be additional entries here.

b. Select **Gen AI studio › AI asset endpoints** and then select the **MCP servers** tab.

Verify that there are no MCP servers configured for use by Gen AI studio playgrounds.

c. In the left pane, expand the **Develop & train** category, which contains entries named **Feture store**, **Pipelines**, **Jobs**, and **Evaluations**.

d. In the toolbar, to the top right, click the **Applications** icon, which is close to the **Question** icon, and verify that there is **not** an entry named **MLFlow**.

After completing these steps, you have verified that you have a functional RHOAI instance with a pre-deployed model, which is available only for internal access, and that you can submit prompts to the model using an RHOAI playground.

In addition, you have also verified that MLFlow is not installed on your RHOAI instance, and that there are no MCP servers deployed.

What’s next

In the next chapter, you will learn how to deploy MLFlow on Red Hat OpenShift AI to enable experiment tracking and model management capabilities.

Deploying MLFlow for Development

Goal

Deploy a development environment with MLFlow

This chapter guides you through deploying MLFlow in Red Hat OpenShift AI for development purposes. You'll learn about MLFlow's architecture, how it integrates with RHOAI, and gain hands-on experience deploying and verifying your MLFlow instance.

MLFlow Architecture and RHOAI Integration

*Estimated reading time: **5 minutes**.*

Objective

Understand MLFlow's architecture and how it integrates with Red Hat OpenShift AI.

> Not started yet.

MLFlow Components

Lorem ipsum

RHOAI Integration

Lorem ipsum

Development vs Production Deployments

Lorem ipsum

What's Next

Now that you understand MLFlow's architecture and integration with RHOAI, the next section provides hands-on experience deploying MLFlow in a development environment.

Lab: Deploy MLFlow on Red Hat OpenShift AI

Objective

Deploy and verify a development instance of MLFlow in Red Hat OpenShift AI.

> Pending review

Before you begin

You need access to the predeployed Red Hat OpenShift AI (RHOAI) instance provided in your lab environment. Ensure you have:

- Access credentials for the RHOAI dashboard
- A web browser to access the RHOAI dashboard
- Access credentials for the OpenShift web console

Instructions

You will enable MLFlow on your Data Science Cluster singleton resource and deploy a development instance of the MLFlow server. Then you will create a simple Jupyter notebook to test connectivity to the MLFlow server by means of the MLFlow SDK for Python.

As you perform this activity, you will alternate between three browser tabs:

- The Red Hat OpenShift web console
- The Red Hat OpenShift AI (RHOAI) dashboard
- A Jupyter notebook from a RHOAI workbench.

1. Enable MLFlow on RHOAI.

    a. Open the OpenShift web console and log in as a user with `cluster-admin` rigts.

    b. In the left navigation pane, select **Ecossystem › Installed Operators**. Then click **Red Hat OpenShift AI**.

    c. In the Red Hat OpenShift AI operator page, select the **Data Science Cluster** tab. Then click the **default-dsc** resource.

    d. In the `default-dsc` resource page, select the **YAML** tab. In the text editor, search for the `spec.components` field, and set its `mlflowoperator.managementState` field to `true`.

    ```
    ...
    spec:
      components:
        mlflowoperator:
          managementState: Managed
        kserve:
          managementState: Managed
          rawDeploymentServiceConfig: Headless
    ...
    ```

    e. Click **Save** and wait until you see a confirmation message similar to:

    ```
    alert:default-dsc has been updated to version 1186240
    ```

    f. If you get an error message, fix the errors on your YAML and try saving again.

    g. Alternatively, you could run the following `oc patch` command, which is provided by the RHOAI product docs:

    ```
    $ oc patch datasciencecluster default-dsc --type=merge \
    ```

```
-p
'{"spec":{"components":{"mlflowoperator":{"managementState":"Managed"}}}}'
datasciencecluster.datasciencecluster.opendatahub.io/default-dsc patched
```

2. Configure your MLFlow instance.

   a. In the left navigation pane, select **Home › API Explorer**. Then type `MLflow` in the search field. You will see three resource types, click on **MLflow**.

   b. In the MLflow Resource details page, select the **Instances** tab and click **Create MLflow**.

   c. In the YAML text editor, delete all of its sample contents and type the following, which corresponds to the example from Minimal development or test deployment in product docs.

```
apiVersion: mlflow.opendatahub.io/v1
kind: MLflow
metadata:
  name: mlflow
  namespace: redhat-ods-applications
spec:
  storage:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 10Gi
  backendStoreUri: "sqlite:////mlflow/mlflow.db"
  artifactsDestination: "file:///mlflow/artifacts"
  serveArtifacts: true
```

   d. Click **Create** and, in the MLflow details page, wait until the `Available` condition is `True` and the `Progressing` condition is `False`.

3. Verify MLFlow is available from RHOAI

   a. Open your RHOAI dash board. If it is already open, refresh your browser window.

   b. In the left navigation pane, expand **Develop & train**. Notice the new item named **Experiments (mlflow)** and click it.

   c. In the Project combo box, type `my` and select the `my-first` model project.

   d. Click **Create Experiment** and type `dummy experiment` as its name.

   e. Verify that you see the Experiments page for `dummy experiment` but there is no data on any of its pages, such as **Traces**, **Sessions**, or **Prompts**.

   f. Creating an empty experiment is sufficient to prove that your MLflow instance can connect to its database and artifacts storage.

   g. You can, alternatively, open the upstream MLFlow web interface by clicking the applications menu, in the top navigation bar, close to the help icon, and selecting **MLflow**. You should not need it: all funcionality you need for this course is available from the RHOAI dashboard.

4. Test connectivity to MLFlow from a workbench.

    a. In the left navigation pane, click **Projects** and select **All projects** in the projects combo box. Then type `my` in the filter by name field and click `my-first-project` in the list of projects.

    b. In the Projects page for `my-first-model`, select the **Workbenches** tab and click menu:Create workbench.

    c. In the Create workbench page, type `test-mlflow` for the **Name** field and, in the Workbench image field, select the `Jupyter | Minimal | CPU` image.

    d. Leave all other fields with their default values and click **Create workbench**.

    e. In the Workbenches list, wait until the row for the `test-mlflow` workench displays a `Ready` state and then click **test-mlflow** to open its Jupyter Lab in a new browser tab.

    f. Click the **Python 3.12** Notebook icon to create an empty notebook.

    g. In the first cell, type:

```
! pip install mlflow[kubernetes]
```

    h. In the top toolbar from the notebook, click the play icon, with the tooltip **Run this cell and advance**.

    i. Wait until the MLFlow SDK and all its dependencies are loaded from the RHOAI index into the workbench.

    j. After the `pip` command finishes, type the following on the second cell of your notebook:

```
import os
import mlflow

os.environ["MLFLOW_TRACKING_AUTH"]="kubernetes-namespaced"
mlflow.set_tracking_uri("CHANGE_ME")

print(mlflow.list_workspaces())
```

    k. Switch to your RHOAI dasbhoard page, which should still be in the Experiments page. Notice, close to the top left, the **Launch MLflow** link. Right-click it and copy its URL to the clipboard.

    l. Switch to your jupyter lab browser tab and replace, in the second notebook cell, the `CHANGE_ME` text with the URL on your clipboard.

In this example, we are using the *external* URL of the MLFLow server, which depends on the applications domain of your OpenShift Ingress operator. This is the URL you would use for applications running outside of its OpenShift cluster.

> External applications also need to set the `MLFLOW_TRACKING_TOKEN` environment variable with a valid an OpenShift authentication token, for example the one returned by the `oc whoami -t` command.

m. In the top toolbar from the notebook, click the play icon. The results should an array containing a single `Workspace` object named `my-first-model`.

   This notebook takes advantage of the integration between the RHOAI dashboard and its managed instance of the MLFlow server. It connects using the service account of the `test-mlflow` workbench, which is authorized to access the MLFlow workspace corresponding to its `my-first-model` project.

n. If you wish, you can remove the unnamed notebook, or if you prefer you can save it, rename it to `test.ipynb`, and reuse for the next activity. Whatever your preference, do NOT delete the `test-mlflow` workbench because you will keep using it in the next activity.

After completing these steps, you have verified that you have a minmal but funcional MLFLow instance, to which you can connect from applications running inside your RHOAI cluster and store data from your experiments.

What's next

Now that you have a working MLFlow deployment, the next chapter explores how to collect traces from agentic applications using the MLFlow SDK.

Collecting Traces from Agentic Applications

Goal

Collect traces from an agentic application using MLFlow

This chapter teaches you how to use the MLFlow SDK to collect traces from agentic applications. You'll learn why tracing is essential for troubleshooting agents, how to organize traces in experiments and runs, and gain hands-on experience adding MLFlow tracing to your code.

Tracing Agentic Applications

*Estimated reading time: **5 minutes**.*

Objective

Understand why tracing is essential for agentic applications and how MLFlow organizes trace data.

> Not started yet.

Why Developers Need Traces

Lorem ipsum

MLFlow SDK and Trace Calls

Lorem ipsum

Experiments and Runs

Lorem ipsum

Significant Events in Traces

Lorem ipsum

What's Next

Now that you understand tracing concepts, the next section provides hands-on experience modifying an agent to record traces using the MLFlow SDK.

Lab: Modify an Agent to Record Traces

Objective

Add MLFlow tracing to an agentic application and visualize the generated traces.

> Pending review

Before you begin

You need access to the predeployed Red Hat OpenShift AI (RHOAI) instance provided in your lab environment, which was already configured by the previous lab with a development instance of the MLFlow server and an RHOAI workbench.

Ensure you have:

- Access credentials for the RHOAI dashboard
- A web browser to access the RHOAI dashboard
- Access credentials for the OpenShift web console

Instructions

You will create a simple chat application using a workbench, validate it is working, and instrument the application with calls to MLFlow. Then, you will review traces generated by a couple test runs of the instrumented application.

As you perform this activity, you will alternate between three browser tabs:

- The Red Hat OpenShift web console
- The Red Hat OpenShift AI (RHOAI) dashboard
- A Jupyter notebook from a RHOAI workbench.

Once you open one of these tabs, keep it open because you may return to it later.

1. Create a Jupyter notebook for you chat application.

   a. Open the RHOAI dashboard and log in using the `admin` user.

      You do not need administrator privileges to complete this lab, but the demo instance is not configured with any user other than `admin`.

   b. In the left navigation pane, select **Projects** and, in the **Project** field, select **All projects**. Then type `my` and, in the list of projects, select **my-first-model**.

   c. In the projects page for `my-first-model`, select the **Workbenches** tab. Click the **test-mlflow** workbench, which you created in the previous lab, to open its jupyter lab in a new browser tab.

   d. In the jupyter lab, select the **Launcher** tab and click the **Python 3.12** icon to create a new jupyter notebook.

   e. In the left navigation pane, right click the **Untitled.ipynb** file, which is your new notebook, and select **Rename**. Then type `basic-chat` as its new file name.

   f. Alternatively, you can open and reuse the `test.ipnyb`, which you created in the previous lab.

2. Insert the code for a simple chat application into your notebook.

   a. In the first cell, use a `pip` command to install the OpenAI client API:

```
! pip install openai
```

b. In the top toolbar from the notebook, click the **play** icon, with the tooltip **Run this cell and advance**.

c. Wait until the OpenAI library and its dependencies are loaded from the RHOAI index into the workbench.

d. In the second cell, set some variables designed to adapt your application to its running environment.

```
BASE_URL = "CHANGE_ME"
model_name = "llama-32-3b-instruct"
```

e. Switch to the RHOAI dashboard and select, in the left navigation pane, **AI hub › Models**, then select the **Deployments** tab. In the row for the `llama-32-3b-instruct` model, click **Internal endpoint** to display a pop-up with the access URL for its model server, and click the **Copy** icon in the pop-up.

f. Switch to your Jupyter notebook and replace the `CHANGE_ME` text with the URL on your clipboard and type `/v1` at the end of the URL to make it a valid OpenAI API endpoint.

g. Continue editing the second cell and set a system and a user prompt.

```
input_text = """
What's the difference between RHEL and CentOS?
"""
system_instructions = """
You are a helpful AI assistant.
You are designed to answer questions in a concise and professional manner.
"""
```

h. Add to the cell code to retrieve the authentication token from its Kubernetes service account and connect to the model server using the OpenAI API.

```
import os
from openai import OpenAI

# there should be an easier way of getting the service account token
try:
    with open("/var/run/secrets/kubernetes.io/serviceaccount/token", "r") as f:
        AUTH_TOKEN = f.read().strip()
except FileNotFoundError:
    # Fallback if running outside a workbench or using a manually generated
cluster token
    AUTH_TOKEN = OCP_TOKEN

client = OpenAI(
```

```
    base_url=BASE_URL,
    api_key=AUTH_TOKEN
)
```

i. Add code to submit the prompt to the model and display tits response.

```
try:
    response = client.chat.completions.create(
        model=model_name,
        messages=[
            {"role": "system", "content": system_instructions},
            {"role": "user", "content": input_text}
        ],
        max_tokens=256,
        temperature=0.7
    )

    # Print the output
    print(response.choices[0].message.content)

except Exception as e:
    print(f"An error occurred: {e}")
```

3. Run your chat application to verify it is working.

   a. It is a good time to save you code changes. In the top toolbar of your notebook, click the **disk** icon, having tooltip **Save and create checkpoint**.

   b. Now that your second notebook cell contains the entire code of the chat application, click the **play** icon in the top toolbar of your notebook.

   c. Hopefully, your output is correct and useful. You do know what makes RHEL different than CentOS, right? If you get a hallucinated response, just continue with the activity. You will have a chance of seeing a good response later in this lab.

   Follows an example of a good response we got while testing these instructions:

```
Red Hat Enterprise Linux (RHEL) and CentOS are both popular Linux distributions,
but they have distinct differences:

1. Origin:
   - RHEL is a proprietary Linux distribution developed and maintained by Red
Hat, Inc.
   - CentOS is a community-supported Linux distribution, based on RHEL, and is
also known as "Community Edition" or "Free Community Supported Edition".

2. Support and Licensing:
   - RHEL has paid support and licensing, which provides access to Red Hat's
customer support, updates, and security patches.
   - CentOS is free and open-source, but it relies on the community for support
```

```
and updates.

3. Update Model:
   - RHEL follows a traditional release model, where new versions are released
every 2-3 years, and updates are provided through subscription.
   - CentOS follows a rolling release model, where new versions are released
based on the RHEL version, and updates are provided through the community.

4. Binary Compatibility:
   - RHEL and CentOS are binary compatible, meaning that software compiled for
RHEL will run on CentOS, and vice versa.

In summary, RHEL is a paid, enterprise-focused distribution, while CentOS is a
free, community-supported distribution based on RHEL.
```

4. Instrument your chat application with MLFlow tracing.

   a. Edit the first cell to also install the MLFlow SDK for Python.

   ```
   ! pip install openai mlflow[kubernetes]
   ```

   b. Click the **play** icon in the toolbar and wait until the MLFlow library and its dependencies are loaded from the RHOAI index into the workbench.

   c. Edit the second cell to add a few variables, which point to the MLFlow server from your RHOAI instance.

   ```
   BASE_URL = "http://llama-32-3b-instruct-predictor.my-first-
   model.svc.cluster.local:8080/v1"
   model_name = "llama-32-3b-instruct"

   MLFLOW_TRACKING_URL = "https://mlflow.redhat-ods-
   applications.svc.cluster.local:8443"
   MLFLOW_EXPERIMENT = "basic-chat"
   ```

   In this example, we are using the *internal* URL of the MLFLow server, which is fixed. You can use this URL only with applications running on the same OpenShift cluster.

   d. Add code to configure the connection to the MLFlow server, right before the code which gets your kubernetes service account token.

   ```
   import os
   from openai import OpenAI
   import mlflow

   os.environ["MLFLOW_TRACKING_AUTH"]="kubernetes-namespaced"
   mlflow.set_tracking_uri(MLFLOW_TRACKING_URL)
   ```

```
mlflow.set_experiment(MLFLOW_EXPERIMENT)

# there should be an easier way of getting the service account token
try:
    with open("/var/run/secrets/kubernetes.io/serviceaccount/token", "r") as f:
        AUTH_TOKEN = f.read().strip()
except FileNotFoundError:
    # Fallback if running outside a workbench or using a manually generated
cluster token
    AUTH_TOKEN = OCP_TOKEN
```

e. Add code to start MLFlow autologin, right before the code which connects to the model server.

```
mlflow.openai.autolog()
client = OpenAI(
    base_url=BASE_URL,
    api_key=AUTH_TOKEN
)
```

5. Run your chat application to verify it continue working after changes.

   a. It is again a good time to save you code changes. In the top toolbar of your notebook, click the **disk** icon, having tooltip **Save and create checkpoint**.

   b. Ensure the second cell is selected and click the **play** icon in the top toolbar of your notebook.

   c. Hopefully, your output is again correct and useful, and probably have small differences in wording compared to your previous output. You can ignore warnings which suggest updating jupyter and ipywidgets. More important, check that you see a message stating that a new experiment was created, similar to the following partial example ouput.

```
...
2026/07/01 19:47:59 INFO mlflow.tracking.fluent: Experiment with name 'basic-
chat' does not exist. Creating a new experiment.

RHEL (Red Hat Enterprise Linux) and CentOS are both Linux distributions, but
they have some key differences:
...
```

6. View the trace in the RHOAI dashboard.

   a. Switch to your brower tab on the RHOAI dashboard, which should be already in the Experiments page, and notice there is a new item, besides the `dummy experiment` created by the previous lab.

   b. Click the **basic-chat** experiment, which was created when you re-run your notebook after making changes.

   c. You will see a page displaying many statistics and graphics about the performance of your experiments. Each data point corresponds to one time you run your chat application.

d. In the left pane of the Experiments page, without leaving the `basic-chat` experiments page, click **Traces**.

e. You will see a page containing one trace per row. Each row displays a trace ID, its request, response, and some performance metrics.

f. Click on either the trace id or its prompt to see a summary of the trace. Click on the links **Show 1 more** and similar on the input and output pane to expand the details of the inputs sent to the model server and the outputs received from it.

   From the information in a trace, it is possible to reproduce the request and compare its outputs between different attempts.

g. Click the **x** icon to close the trace summary page and return to the list of traces from the `basic-chat` experiment.

7. Generate a second trace from the chat application.

   We could repeat the same prompt multiple times, to verify variations in output and in performance metrics but, to make it more interesting, change the system prompt so you see a significative different in the responses your application produces.

   a. Switch to your jupyter notebook and append more instructions to its system prompt:

   ```
   input_text = """
   What's the difference between RHEL and CentOS?
   """
   system_instructions = """
   You are a helpful AI assistant.
   You are designed to answer questions in a concise and professional manner.
   Answer with one to three sentences.
   """
   ```

   b. Ensure the second cell is selected and click the **play** icon in the top toolbar of your notebook.

   c. There should be no messages from MLFlow, because it is using an existing experiment. And you should get a shorter response, which hopefully is still good, similar to the following example:

   ```
   RHEL (Red Hat Enterprise Linux) and CentOS are both Linux distributions, but
   they have different licensing and support models.
   RHEL is a commercial distribution supported by Red Hat, while CentOS is a
   community-supported distribution that mirrors RHEL.
   CentOS is based on RHEL, but no longer receives security updates from Red Hat,
   making RHEL the more secure choice.
   ```

   If you see a button **Expand MLFlow Trace**, do **not** click it. The visual integration between MLFlow and RHOAI workbenches is not yet working with RHOAI 3.4.

8. View the additional traces in the RHOAI dasbhoard.
   a. Switch to your RHOAI dashboards, which should be still in the Traces list of the `basic-chat` experiment. You may need to refresh your browser window to view the second trace.
   b. Click the trace ID of the first item, which is the most recent trace, so it corresponds to the latest time you run your notebook.
   c. Verify that the trace summary shows the latest, longer system prompt on its inputs, and the latest, shorter response on its outputs. Do **not** close the trace summary page.
9. Register a human assessment, simulating what a real-world chat application might do when a user clicks either a thumbs up or a thumbs down icon.
   a. Click **Show assessments** to display the Assessments pane, to the right of the trace summary page.
   b. Click **Add feedback** and select **Human feedback**.
   c. Type `good response` as the Assessment Name. Leave the Data Type as `Boolean` and Value as `True` and click **Create**.
   d. Click the **x** icon of the trace summary pane. There is also an **x** icon on the Assessments page, which is **not** the one you want.
   e. Verify that the traces page of the `basic-chat` experiment now displays an Assesments column, and a summary of the distribution of values for the `good response` assesment among all traces.
   f. Click the trace ID of the previous trace, from before you changed the system prompt. Add an human assessment using the same name but with a value of `False`. Close the summary page of that trace and see how the summary changes.

After completing these steps, you have executed a simple chat application with MLFLow tracing enabled. You also compared traces from multiple runs of the chat application and recorded human assessments of its responses.

What's next

Now that you've added basic tracing to an agent, the next section explores how to trace MCP tool calls to capture more detailed information about agent behavior.

Lab: Trace MCP Tool Calls from an Agent

Objective

Deploy an MCP server and capture its tool invocations in MLFlow traces.

Pending review.

Before you begin

You need access to the predeployed Red Hat OpenShift AI (RHOAI) instance provided in your lab environment, which was already configured by the second lab with a development instance of the MLFlow server and an RHOAI workbench.

Ensure you have:

- Access credentials and the URL for the RHOAI dashboard
- A web browser to access the RHOAI dashboard
- Access credentials and the URL for the OpenShift web console

Instructions

You will deploy an MCP server which allow instrospection of the state of an OpenShift cluster and its resources. Then, you will create a simple agent which uses that MCP server to assess the state of workloads running on your OpenShift cluster. Finally, you will interpret traces from the agent to assess if it is able to make use of tools from the MCP server.

As you perform this activity, you will alternate between three browser tabs:

- The Red Hat OpenShift web console
- The Red Hat OpenShift AI (RHOAI) dashboard
- A Jupyter notebook from a RHOAI workbench.

Once you open one of these tabs, keep it open because you may return to it later.

1. Deploy the MCP Server for Red Hat OpenShift with restricted privileges, by using its helm chart and the OpenShift Ecosystem Catalog.
    a. Open the OpenShift web console and log in using the `admin` user. In the left navigation menu, select **Ecosystem › Software Catalog**. Scroll down to find the **Type** heading and click **Helm charts**.
    b. Scroll up to the **All items** field and type `mcp`. Then click the entry **Red Hat OpenShift Mcp Server** and, on its details pane, click **Create**.
    c. In the **Create Helm Release** page, click the **Project** field, which probably displays the `default` project, and click **Create Project**. Type `mcp-servers` as the **Name** field and click **Create**.
    d. In the **Create Helm Release** page, delete all contents from its text editor and paste the following configuration, which gives the MCP server read-only access to all resources int the cluster, but disables external access.

    ```
    ingress:
      enabled: false
    rbac:
      extraClusterRoleBindings:
        - name: use-view-role
          roleRef:
            name: view
            external: true
    ```

    e. Click **Create**. You should be now in the **Workloads › Topologu** page of the `mcp-servers` project, which displays a Helm icon for the MCP server deployment. Click the Helm icon to display its workload details page, to the right, and wait until its only pod is runnig.

2. Test the MCP server using the Gen AI studio

   a. Create a config map which provides the Gen AI studio with a list of available MCP servers.

      In the left navigation pane, select **Workloads › ConfigMaps** and, in the **Project** field at the top, select the `redhat-ods-applications` project.

   b. Click **Create ConfigMap** and, in the **Create Configmap** page, type `gen-ai-aa-mcp-servers` as the **Name** field.

   c. Type the following as the **Value** field and click **Create**.

```
{
      "url": "http://redhat-openshift-mcp-server.mcp-
servers.svc.cluster.local:8080/mcp",
      "description": "Red Hat OpenShift MCP server"
}
```

   d. Verify that the Gen AI studio can connect to the MCP server.

      Open your RHOAI dashboard and login as the `admin` user. In the left navigation pane, select **Gen AI studio › AI assets endpoints** and select the **MCP servers** tab. It should list the `OpenShift-MCP-Server`, as configured by the config map, with the `Active` status, which means the Gen AI studio was able to retrieve the tools list from the MCP server.

   e. Test the MCP server tools using a playground.

      In the left navitation pane, select **GenAI studio › Playground**. Then type `my` in the **Project** field and select **my-first-project**. Click **Create playground**. In the **Configure playground** form, leve the model checked and click **Create**.

   f. Wait while the Gen AI studio deploys a llama stack server for your playground. It displays a confirmation popup when done and enters the **Playground** page,

   g. In the **Configure** pane of the playground, select the **MCP** tab and check the row with name `OpenShift-MCP-Server`. You should see a popup stating **Connection successful** and the number of tools provided by the MCP server. Click **Save** to select all tools.

   h. In the right pane, click the `Send a message…` text and type a prompt which requires using the MCP server, for example:

```
Which pods are in my-first-model?
```

   i. Type `Enter` to submit the prompt. Hopefully, you will get a correct response, which you can optionally verify by switching to the OpenShit web console and listing pods from the `my-first-model` project.

3. Import the Jupyter notebook for a simple agent.

a. In the left navigation pane, select **Projects** and, in the **Project** field, select **All projects**. Then type `my` and, in the list of projects, select **my-first-model**.

b. In the projects page for `my-first-model`, select the **Workbenches** tab. Click the **test-mlflow** workbench, which you created in the second lab, to open its jupyter lab in a new browser tab.

c. In the left toolbar of your Jupyter Lab, click the **Git Clone** icon.

d. In the **Clone a repo** popup, type `https://github.com/RedHatQuickCourses/rhoai-mlflow-samples.git` as the URI of the remote Git repository and click **Clone**.

e. In the file browser of your Jupyter Lab, enter the `/rhoai-mlflow-samples/traces` directory and then open the `basic-agent.ipynb` notebook.

4. Review the code for a simple agent.

> The expanations here are **not** intended to explain all code in the agentic application, but just its general structure, so you can later read its traces. The code snippets here enable learners to locate each section of code, if they wish to study and use it as a starting point for their agentic applications.

a. In its first cell, notice the use of `--extra-index-url` option for the `pip install` command. It is necessary because the RHOAI index does not include the `openai-agents` library. That way, pip falls back to the comunity index for any module not present in its primary index.

```
! pip install --extra-index-url https://pypi.org/simple openai openai-agents
mlflow[kubernetes]
```

b. In the second cell, notice the variables which set the URLs and connection parameters for the model server, the MCP server, and the MLFlow server. They are all internal URLs, which refer to their respective Kubernetes services.

```
BASE_URL = "http://llama-32-3b-instruct-predictor.my-first-
model.svc.cluster.local:8080/v1"
model_name = "llama-32-3b-instruct"

MCP_SERVER_URL = "http://redhat-openshift-mcp-server.mcp-
servers.svc.cluster.local:8080/mcp"

MLFLOW_TRACKING_URL = "https://mlflow.redhat-ods-
applications.svc.cluster.local:8443"
MLFLOW_EXPERIMENT = "basic-agent"
```

c. Then it sets a user prompt, which should require the MCP server, and a generic system prompt.

```
input_text = """
List the names of all pods in namespace 'my-first-model'
```

```
"""
system_instructions = """
You are a helpful AI assistant.
You are designed to answer questions in a concise and professional manner.
"""
```

d. So far, the code is very similar from previous examples but, after it gets the service account token and eneables MLFlow tracing, the code to connect to the model and submit a prompt is *very* different from the previous chat example.

   Differences start by creating an asynchronous client for the model server:

```
async def main():
    """Main async function to run chat with MCP tools"""

    # Configure custom OpenAI endpoint globally
    custom_client = AsyncOpenAI(
        base_url=BASE_URL,
        api_key=AUTH_TOKEN
    )
```

e. It also creates an asynchronous MCP client:

```
    async with MCPServerStreamableHttp(
        name="Remote MCP Server",
        params={
            "url": MCP_SERVER_URL,
            "timeout": 30,
        },
        cache_tools_list=True,
    ) as mcp_server:
```

f. The agent object refers, implicitly, to the global OpenAI endpoint configured earlier, and also refers explicitly to the MCP client and the system prompt.

```
        agent = Agent(
            name="Assistant",
            instructions=system_instructions,
            model=model_name,
            mcp_servers=[mcp_server],
        )
```

g. An asynchronous runner submits the prompt to the agent and displays its final response.

   The agent loops on the responses received from the model, invoking MCP tools, and sending the output of tool calls back to the model.

This loop, inside the agent object, continues until the agent gets a response from the model which does **not** request another tool call.

```
        # Run the agent
        result = await Runner.run(agent, input_text)

        # Print the final output
        print("Assistant:", result.final_output)
```

h. The final piece of code enables running the asyncronous code from both a standalone application and a Jupyter notebook, which already sets an asynchronous runner for its cells.

```
        try:
            get_ipython()
            # Running in Jupyter - use await directly (top-level await is
supported)
            import nest_asyncio
            nest_asyncio.apply()
            asyncio.run(main())
        except NameError:
            # Not in Jupyter - use asyncio.run() normally
            asyncio.run(main())
```

5. Run the agent to produce a trace.

   a. In the top toolbar from the notebook, click the **fast forward** icon, with the tooltip **Restart the kernel and run all cells**.

   b. It may take a while to download all OpenAI and MLFlow libraries and their dependencies, in the first cell before it actually submits the prompt, in the second cell.

   If all goes well, you will get a correct response, similar to the following example:

```
Connecting to MCP server at http://redhat-openshift-mcp-server.mcp-
servers.svc.cluster.local:8080/mcp...

Found 14 tools from MCP server:

Processing query: List the names of all pods in namespace 'my-first-model'

Assistant: Based on the output, the names of all pods in the namespace 'my-
first-model' are:

1. llama-32-3b-instruct-predictor-6bd8fbdd49-28b9m
2. llama-32-3b-instruct-predictor-6bd8fbdd49-j9dxk
3. llama-32-3b-instruct-predictor-6bd8fbdd49-nkvn9
4. llama-32-3b-instruct-predictor-6bd8fbdd49-vvwgb
5. llama-32-3b-instruct-predictor-6bd8fbdd49-wbgxn
6. lsd-genai-playground-55555b9d99-78zqk
```

```
7. test-mlflow-0
```

The output includes the number of tools from the MCP server as proof that the agent can connect to it, even if the response looks like it didn't.

> There is a higher change of getting an incorrect response, or no reponse at all, compared to the previos example. The later may happen because the agent received too many tool call requests from the model, and decided that the model is lost and cannot produce a final response.

6. View the trace in the RHOAI dashboard.

    a. Switch to the RHOAI dashboard and, in the left navigation pane, select **Develop & train › Experiments (MLflow)**. In the **Project** field, type `my` and select the **my-first-model** project. Then click the **basic-agent** experiment.

    b. In the left pane of the **Experiments** page for the `basic-agent` project, click **Traces**. You should see a trace whose request and reponse correspond to your notebook run. Click it.

    c. In the summary page for the trace, you will see a number of items labeled **AsyncCompletions**. Each of them corresponds to a roundtrip between the agent and the model. If you are lucky, you will see two of them.

    d. Click the **expand/collapse** icon (>) from the first **AsyncCompletions**, but do **not** click the **AsyncCompletions** itself. You will see it contains **Inputs** and **Outputs**.

        The **Inputs** include the user and system prompts, a list of tools, and a number of OpenAI parameters which the sample agent didn't set explicitily, so they took their default values.

        The **Outputs:** include a reques to call the tool named `pods_lists_in_namespace` with arguments `"namespace": "my-first-model"`

    e. Click the **expand/collapse** icon (>) from the second (or latest) **AsyncCompletions**.

        Notice its **Inputs** include everythig you saw from the previous completions in the same trace, including prompts and lists of tools. But it adds the results of the tool call requested by the previous completion.

        It may be hard to read in the trace, but you can recognise similarities with the output of an `oc get pod -o wide` command.

        The **Outputs** should match the final response displayed by your notebook run.

    f. If you got more than two **AsyncCompletions** items, it might be because the model was not able to recognize that the tool call contains the information it needs, and requested a another tool call, maybe with different parameters, maybe of a different tool.

        For more complex prompts, it might be necessary to use the output of a tool call as parameters of another tool call, and many such rounds might be necessary. But you may find that, even with simpler prompts, a model might requests tool calls which make no sense.

g. When you are satisfied that you understand the trace, click the its **close** icon, that is, the **x** to the top right of the trace summary page, to return to the list of traces.

7. (Optional) You may switch to your Jupyter Lab and run the second cell again a few times. Each run produces its own trace, and they may take different numbers of completions to produce its response.

    a. Sometimes the responses will be correct, as in including the sames of all pods and no extra pods, but vary on their wording.

    b. Sometimes you may get an incorrect response, such as `This output indicates that there are no pods in the 'my-first-model' project.`

    c. Because the demo enviroment only supports a small model, with a small context window, you can easily devise a prompt which produces an error, such as lising all pods in the cluster. But you can also find that there are a number of useful prompts this small model can answer correctly and frequently.

    d. You may also try subtle changes in the prompts, for example referring to "the my-first-model project" instead of to a "namespace", or trying to provide a better suited system prompt for an IT operations or a Kubernetes administrator persona.

After completing these steps, you have executed a simple agentic application which produces traces on the MLFlow server in RHOAI. You also interpreted those traces to identify when the model requested tool calls, the specific data returned by the MCP server for each call, and how the data was send back from the agent to the model. Finally, you verified that the non-deterministic nature of large language models (LLMs) impacts their ability of requesting tool calls and using the returned data to produce a response.

What’s next

Congratulations! You’ve completed this course and learned how to deploy MLFlow, collect traces from agentic applications, and visualize trace data. You can now apply these skills to troubleshoot and improve your own agentic applications.